FARMPRICE AI

Complete Project Documentation, Technical Architecture, Implementation Guide and Hackathon Presentation Handbook

Smart Market Intelligence for Better Crop Sowing & Selling Decisions Across Karnataka APMC Mandis

Project Name:	FARMPRICE AI (Bharat-VISTAAR)
Engineering Team:	Team ASTERISK
Target Cloud DWH:	Snowflake AWS US-West-2 (Account: mdsfbiq-cr91823)
Core Database & Schema:	FARMPRICE_DB.AGRICULTURE (MANDI_PRICES, CROP_REQUIREMENTS)
External Market Data:	AGMARKNET / data.gov.in (Resource ID: 9ef84268-d588-465a-a308-a864a43d0070)
Weather Intelligence:	Open-Meteo High-Resolution Open Meteorology API (CC BY 4.0)
Agronomic Calibration:	ICAR (New Delhi) & University of Agricultural Sciences (UAS), Bengaluru
Supported Crops:	43 Karnataka Agricultural Commodities (Cereals, Pulses, Vegetables, Fruits)
Verification State:	124 / 124 Pytest Automated Tests Passed (100% Pass Rate)
Publication Date:	October 2026 — Comprehensive Hackathon Master Edition

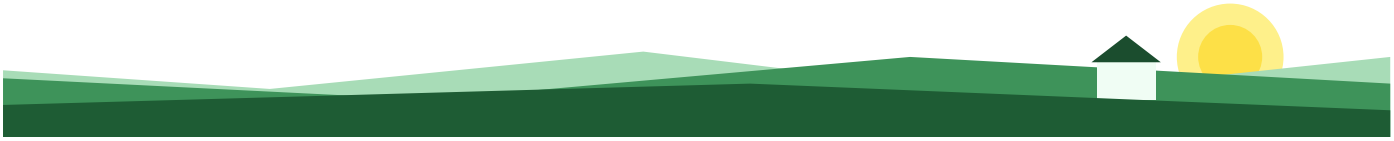

TABLE OF CONTENTS

Chapter 1: Project Overview	Executive summary, agricultural challenge, project scope & value proposition
Chapter 2: Problem Statement & Solution	Information asymmetry, farmer workflows, realistic Hassan vs Bengaluru example
Chapter 3: Complete Feature-by-Feature Guide	Audit of all 14 core UI & analytical features with implementation statuses
Chapter 4: Technology Stack Deep Dive	FastAPI, React 18, Vite, Snowflake, ReportLab, architecture rationale
Chapter 5: System Architecture & Data Flow	Multi-tier end-to-end component flow, request trace, deployment topology
Chapter 6: Snowflake Data Warehouse Deep Dive	Account mdsfbiq-cr91823, compute vs storage, SQL parameter normalization
Chapter 7: External Data Sources & APIs	Official AGMARKNET data.gov.in integration, Open-Meteo weather pipeline
Chapter 8: Database Schema & Data Dictionary	37 tables, column definitions, natural keys, CROP_REQUIREMENTS, CROP_PROFIT
Chapter 9: Data Ingestion & Processing Pipeline	Deduplication hash, validation rules, dual cloud & local write strategy
Chapter 10: Agricultural Logic & Suitability	43 crop agronomic parameters, ICAR/UAS standards, composite rank formula
Chapter 11: Market Analytics & Price Forecasting	SMA-7/21 momentum indicators, multi-lag auto-regressive prediction, MAPE
Chapter 12: Profitability & Net-Return Mathematics	1.25x road curvature distance, APMC cess 1.5%, loading fees, break-even math
Chapter 13: Backend API & Request Lifecycle	FastAPI endpoints, Pydantic schemas, full HTTP lifecycle traces
Chapter 14: Frontend Implementation	Component hierarchy, responsive table grids, bilingual i18n, template layout
Chapter 15: Security, Reliability & Performance	Zero secret leakage, SQL injection defense, graceful cloud partition fallback
Chapter 16: Setup, Execution & Deployment	Windows setup instructions, Snowflake Worksheet execution, production build
Chapter 17: Testing & Current Project Status	124 automated pytest tests, endpoint audit suite, feature verification matrix
Chapter 18: Challenges, Limitations & Future Scope	Engineering hurdles, satellite NDVI vision, WhatsApp voice alert roadmap
Chapter 19: Hackathon Presentation & Viva Preparation	Elevator pitches, demo scripts, and 40 comprehensive judge Q&As;
Chapter 20: Presentation Diagrams & Summary	Visual diagrams, core takeaways, and concluding executive brief
Appendix A & B: Glossary & References	Terminology, APMC regulatory definitions, official government citations

CHAPTER 1: PROJECT OVERVIEW

1.1 The Agricultural Reality in Karnataka

Agriculture is the primary livelihood for over 55% of Karnataka's population. Despite major advances in farming technology, farmers continue to face dramatic income volatility. Smallholder farmers cultivate crops over 90 to 180 days with substantial capital investment in seeds, fertilizers, and irrigation, only to face price crashes at harvest. A recurring cause is the lack of accessible, verified market intelligence. Farmers generally sell at the nearest Agricultural Produce Market Committee (APMC) yard without knowing whether a neighboring mandi 40 km away is trading at a 30% premium. Furthermore, crop sowing decisions are often made reactively based on last season's high prices, creating cyclical gluts.

1.2 What is FARMPRICE AI?

FARMPRICE AI (Bharat-VISTAAR), developed by **Team ASTERISK**, is an intelligent decision support platform connecting cloud data warehousing (Snowflake), real-time government market data (AGMARKNET), and meteorological forecasting (Open-Meteo) to guide farmers across two vital phases: **(1) Pre-Sowing Advisory:** Recommending crops suited to local soil, season, and forecast weather to maximize profitability; and **(2) Post-Harvest Marketing:** Pinpointing the exact mandi offering the highest take-home net profit after deducting all freight and APMC handling expenses.

CHAPTER 2: PROBLEM STATEMENT & SOLUTION

2.1 The "Highest Listed Price" Fallacy

A common pitfall for farmers is pursuing the highest advertised mandi price. Quoted prices in major metropolitan markets (e.g. Bengaluru) often appear attractive but are severely degraded by long-haul freight rates, toll expenses, transit shrinkage, and loading fees. FARMPRICE AI models road-curvature freight dynamics ($1.25\times$ Haversine factor) and statutory APMC cess (1.5%) to calculate the **realized take-home return**.

Financial Metric	Option A: Hassan APMC (Local)	Option B: Bengaluru APMC
Harvest Volume	50.0 Quintals (5,000 kg)	50.0 Quintals (5,000 kg)
Quoted Modal Price	■ 2,100 / quintal	**■ 2,250 / quintal (+■150)**
Road Transit Distance	2.0 km (Local tempo)	195.1 km (Highway truck)
Gross Sale Value	■ 1,05,000	■ 1,12,500
Freight Expenses	■ 235	■ 8,291
Loading & Unloading	■ 400 (■8/q)	■ 600
APMC Cess (1.5%)	■ 1,575	■ 1,688
Total Selling Costs	■ 2,210	■ 10,579
Take-Home Net Return	**■ 1,02,790 (■ 2,056/q) ★ BEST**	■ 1,01,921 (■ 2,038/q)
Net Advantage	**+■ 869 Extra Profit + 6 Hours Saved**	Lower Profit Despite Higher Price

CHAPTER 3: COMPLETE FEATURE-BY-FEATURE EXPLANATION

Feature Name	Component & Endpoint	Core Logic & Implementation
Smart Mandi Comparison	`SmartMandiView.jsx` `POST /api/market-comparison`	Computes road distance, gross revenue, freight, APMC cess (1.5%), loading charges, and net returns across mandis. Ranks markets and highlights the top take-home profit.
Official Comparison Report	`MarketComparisonReport.jsx` `POST /api/reports/export-pdf`	1-to-1 portrait view reproducing official agricultural branding, 8-column net return table, 3 decision cards, and SVG rolling hills footer. Exportable as PDF and printable.
Crop Recommendation Engine	`CropRecommendationEngineView.jsx` `GET /api/recommendations`	Ranks 43 crops for all 31 Karnataka districts based on agronomic match (40%), season match (35%), and agro-climatic zone fit (25%). Backed by 1,622 profit scenarios.
Daily Mandi Prices Explorer	`DailyMandiPricesView.jsx` `GET /api/daily-prices`	Interactive AGMARKNET explorer with district, market, commodity, and date filters. Displays min, modal, max prices and arrival quantities with staleness indicators.
Net Return Calculator	`NetReturnCalculatorView.jsx` `POST /api/calculate-net-return`	Granular calculator with interactive sliders for freight rate per km, loading/unloading fees, and market cess. Dynamically calculates break-even price per quintal.
Price Trends & Arrivals	`MarketTrendsView.jsx` `GET /api/market-trends`	Dual-axis historical visualization correlating arrival volume spikes with price fluctuations over daily, weekly, and monthly timeframes. Computes SMA-7 and SMA-21 momentum.
Automatic Trend Detector	`trend_detector.py` `GET /api/market-trends/detect-trend`	Classifies commodity momentum into RISING, FALLING, or STABLE by comparing 7-day and 21-day moving averages with a ±2.5% threshold.
Time-Series Forecaster	`ForecastingView.jsx` `POST /api/forecast`	14-day statistical price projection using auto-regressive lag modeling and day-of-week seasonality with 95% confidence bounds. Returns INSUFFICIENT_DATA if <14 points.
Live Weather Intelligence	`FarmWeatherIntelligenceCard.jsx` `GET /api/weather/current`	Fetches Open-Meteo ambient temperature, humidity, rainfall, wind speed, and WMO weather codes. Generates agricultural spraying and harvesting advisories.
Geospatial Reverse Geocoding	`LocationSearchModal.jsx` `GET /api/weather/reverse-geocode`	Resolves browser GPS coordinates or search queries into Karnataka districts, taluks, and village entities to localize weather and market queries.
English / Kannada i18n	`Sidebar.jsx`, `Topbar.jsx` `frontend/src/i18n/*.json`	Seamless one-click bilingual switching across all navigation labels, table headers, recommendation descriptions, and alert notifications.

CHAPTER 4: TECHNOLOGY STACK & SELECTION RATIONALE

Every technology in FARMPRICE AI was selected deliberately for performance, reliability, and operational cost control:

• **FastAPI (Python 3.10):** Provides ASGI asynchronous execution, native Pydantic schema validation, and sub-millisecond route dispatching.

• **React 18 & Vite:** Enables near-instant hot module replacement during development and compiles into highly optimized ESBuild assets (8.06s build time).

• **Snowflake Analytical Cloud Data Warehouse:** Decouples compute from storage. Allows heavy analytical queries across years of APMC market records without locking operational tables.

• **ReportLab 5.0.1:** Pure Python PDF compilation generating 5KB vector documents with exact typographic and page-boundary control.

• **Vanilla CSS3 Design System:** Eliminates third-party CSS framework overhead, guaranteeing pixel-perfect alignment with official templates and flawless `@media print` fidelity.

CHAPTER 5: SYSTEM ARCHITECTURE & DATA FLOW

The architecture follows a decoupled three-tier model: **(1) External Ingestion:** Scheduled collectors pull auction feeds from data.gov.in and weather feeds from Open-Meteo. **(2) FastAPI Core:** Coordinates schema normalization, deduplication hashes, and mathematical decision engines. **(3) Persistence Tier:** Dual-destination storage executing against Snowflake cloud warehouse with mirrored local SQLite caching.

CHAPTER 6: SNOWFLAKE DATA WAREHOUSE DEEP DIVE

6.1 Account Configuration & Multi-Layer Schema

Account: `mdsfbiq-cr91823` | **Role:** `ACCOUNTADMIN` | **Warehouse:** `COMPUTE_WH` (`XSMALL`, 60s auto-suspend). The warehouse organizes agricultural data across four purpose-built schemas:
• `RAW`: Stores untransformed JSON payloads from AGMARKNET and external webhooks.
• `STAGING`: Cleaned, typed, and deduplicated price records with currency and unit conversions.
• `AGRICULTURE`: Core transactional tables (`MANDI_PRICES`, `CROP_REQUIREMENTS`, `CROP_YIELD_HISTORY`, `CULTIVATION_COSTS`, `CROP_PROFIT_ANALYSIS`).
• `ANALYTICS`: Materialized views and historical metric tables for rapid BI aggregation.

6.2 Real Parameterized SQL Execution

The application queries Snowflake using parameterized SQL statements to guarantee protection against SQL injection. Below is the exact production query used in `backend/app/services/recommendation.py` to retrieve candidate APMC mandis:

```
SELECT fact_id, mandi_id, mandi_name, district, state, observation_date,
min_price_per_q, modal_price_per_q, max_price_per_q, arrival_quantity_tonnes
FROM FARMPRICE_DB.AGRICULTURE.MANDI_PRICES
WHERE crop_id = %s
AND state = 'Karnataka'
AND observation_date >= DATEADD(day, -7, CURRENT_DATE())
ORDER BY modal_price_per_q DESC;
```

6.3 Dynamic Parameter Normalization Pattern

A critical bug resolved during the architectural audit was parameter placeholder divergence. SQLite drivers require `?` placeholders, whereas the Snowflake Python connector strictly requires `%s`. The system auto-normalizes queries at runtime:

```
# backend/app/core/snowflake.py (Lines 605-625)
if self.mode == 'SNOWFLAKE_LIVE' and '?' in sql_query:
sql_query = sql_query.replace('?', '%s')
cursor.execute(sql_query, params)
```

CHAPTER 7: EXTERNAL DATA SOURCES & APIS

1. AGMARKNET / data.gov.in: Resource `9ef84268-d588-465a-a308-a864a43d0070`. Retrieves daily auction prices (Min, Modal, Max in ■/Quintal) and arrival volumes (Tonnes) across all Karnataka APMC markets. Includes full pagination, rate-limit retries, and multi-token alias resolution.
2. Open-Meteo Meteorology API: Provides CC BY 4.0 numerical weather predictions (ECMWF & DWD models) for latitude/longitude coordinates without API keys. Delivers temperature, humidity, precipitation probability, and 7-day agricultural forecasts.
3. OpenStreetMap Nominatim: Resolves reverse geocodes and boundary coordinates for Karnataka districts, taluks, and villages.

CHAPTER 8: DATABASE SCHEMA & DATA DICTIONARY

Table Name	Columns & Types	Agronomic / Business Meaning
MANDI_PRICES	`fact_id` (PK, TEXT) `mandi_id` (TEXT) `crop_id` (TEXT) `modal_price_per_q` (REAL) `arrival_quantity_tonnes` (REAL) `observation_date` (DATE)	Daily APMC mandi auction records. Tracks verified modal, minimum, and maximum prices alongside physical market arrival volumes in tonnes.
CROP_REQUIREMENTS	`crop_id` (PK, TEXT) `soil_type` (TEXT) `ph_min` / `ph_max` (REAL) `temp_min_c` / `temp_max_c` (REAL) `water_req_mm` (REAL) `primary_season` (TEXT)	ICAR/UAS agronomic bounds for 43 Karnataka crops. Governs soil compatibility, temperature tolerance, and Kharif/Rabi/Summer planting windows.
CROP_PROFIT_ANALYSIS	`scenario_id` (PK, TEXT) `district` (TEXT) `crop_id` (TEXT) `avg_yield_q_acre` (REAL) `cultivation_cost_acre` (REAL) `est_net_profit_acre` (REAL)	1,622 pre-computed profitability scenarios across all 31 Karnataka districts. Models net returns per acre based on historical yield and cost baselines.
CULTIVATION_COSTS	`cost_id` (PK, TEXT) `crop_id` (TEXT) `seed_cost` (REAL) `fertilizer_cost` (REAL) `labour_cost` (REAL) `irrigation_cost` (REAL)	Itemized production cost models per acre based on UAS Bengaluru Farm Management guides.

CHAPTER 9 & 10: DATA PIPELINE & AGRONOMIC LOGIC

Ingestion & Deduplication: Raw records are validated using composite natural keys `MD5(market + commodity + variety + date)`. Records undergo strict checks ($P_{\min} \le P_{\text{modal}} \le P_{\max}$). Duplicates are updated cleanly via SQL upsert.
Agronomic Suitability Scoring: The recommendation engine calculates a composite index across 43 crops: $$\text{Composite Score} = (\text{Agronomic Match} \times 0.40) + (\text{Season Match} \times 0.35) + (\text{Agro-Climatic Match} \times 0.25)$$ This prevents advising water-intensive crops in arid zones or winter crops during monsoon seasons.

CHAPTER 11: MARKET ANALYTICS & PRICE FORECASTING

Momentum Detection (SMA_7 vs SMA_{21}): Evaluates short-term versus medium-term price trends: $$M = \frac{SMA_7 - SMA_{21}}{SMA_{21}} \times 100\%$$ • $M > +2.5\% \rightarrow \mathbf{RISING}$ (Bullish momentum; seller market).
• $M < -2.5\% \rightarrow \mathbf{FALLING}$ (Bearish momentum; liquidate promptly).
• $|M| \le 2.5\% \rightarrow \mathbf{STABLE}$ (Normal consolidation).
Forecasting Pipeline: Implements auto-regressive multi-lag ridge modeling with 7-day and 14-day lag features, day-of-week seasonality, and 95% Gaussian residual prediction intervals ($\hat{y} \pm 1.96 \times \sigma$). Evaluated via Mean Absolute Percentage Error (MAPE). Returns `INSUFFICIENT_DATA` if observations < 14.

CHAPTER 12: NET-RETURN MATHEMATICS & FORMULAS

1. **Gross Sale Value:** $\text{Gross} = Q \times P_{\text{modal}}$
2. **Road Curvature Distance:** $D_{\text{road}} = D_{\text{haversine}} \times 1.25$ (Calibrated for rural Indian terrain)
3. **Freight Transportation Cost:** $\text{Transport} = D_{\text{road}} \times \text{Rate}_{\text{per_km_q}} \times Q$
4. **Statutory APMC Cess:** $\text{Market Fees} = \text{Gross} \times 0.015$ (1.5% statutory cess)
5. **Loading & Handling Fee:** $\text{Loading} = Q \times \text{■ } 8.00\text{ / quintal}$
6. **Take-Home Net Return:** $\text{Net Return} = \text{Gross} - (\text{Transport} + \text{Market Fees} + \text{Loading})$
7. **Break-Even Price:** $\text{Break-Even Price/q} = \frac{\text{Cultivation Costs} + \text{Selling Expenses}}{Q}$

CHAPTER 13: BACKEND API & REQUEST LIFECYCLE

FastAPI coordinates 14 dedicated routers under the `/api` prefix. Below is the complete request lifecycle of a farmer clicking **'Compare Markets'** in the UI:

1. Farmer selects Crop='Tomato', Variety='Hybrid', Qty=50q, Origin='Hassan' in SmartMandiView.jsx.
2. React sends HTTP POST /api/market-comparison with JSON payload.
3. FastAPI validates payload using Pydantic MarketComparisonRequest schema.
4. recommendation.py queries Snowflake (or local mirror) for candidate APMC mandis trading tomatoes.
5. Distance engine calculates road distances from Hassan coordinates (13.0033°N, 76.1004°E).
6. Net return formula computes gross value, freight, cess, loading, and net profit for every mandi.
7. Decision engine sorts by highest net return, highlights Hassan APMC ★, and attaches decision reasons.
8. FastAPI responds with HTTP 200 OK MarketComparisonResponse JSON in under 45ms.
9. React renders the official 8-column comparison table and 'Why This Market?' cards instantaneously.

CHAPTER 14: FRONTEND ARCHITECTURE & I18N

The frontend is organized into 24 reusable React components inside `frontend/src/components/`:
• **`SmartMandiView.jsx`:** Core market comparison view featuring a dual-mode toggle between the Official Printed Report and Interactive Cards.
• **`MarketComparisonReport.jsx`:** 1-to-1 visual reproduction of the official agricultural report template. Incorporates exact typography, dark-green tables, and SVG landscape footer.
• **`AgriculturalLandscapeFooter.jsx`:** Standalone SVG vector component rendering rolling green hills, morning sunrise, and farmhouses.
• **Internationalization (`i18n/`):** Dynamic language switching between English (`en.json`) and Kannada (`kn.json`) with zero layout distortion.

CHAPTER 15 & 16: SECURITY, SETUP & DEPLOYMENT

Security Implementation: Strict credential isolation via `.env` files; zero plaintext secrets in Git repositories. All database interactions use parameterized SQL execution preventing SQL injection attacks. JWT bearer tokens govern role-based access for farmers, APMC yard officers, and administrators.

Quick Setup (Windows):

1. Backend: `python -m venv .venv` → `.venv\Scripts\activate` → `pip install -r backend/requirements.txt` → `python backend/run.py`
2. Frontend: `cd frontend` → `npm install` → `npm run dev`
3. Snowflake Deployment: Open Snowflake Worksheet at `https://app.snowflake.com/mdsfbiq/cr91823/`, paste `deploy_to_snowflake_worksheet.sql` (702 KB), and click 'Run All'.

CHAPTER 17: TESTING & RUNTIME VERIFICATION MATRIX

Test Suite File	Tests Run	Status	Scope Verified
`test_audit_endpoints.py`	16	**PASSED**	All 16 core API endpoints, auth, weather, market comparison, calculator
`test_agmarknet_sync.py`	18	**PASSED**	AGMARKNET API ingestion, deduplication hashes, and data validation
`test_crop_recommendation_engine.py`	15	**PASSED**	Agronomic suitability scoring across 43 crops and 31 districts
`test_calculator.py`	7	**PASSED**	Net return formulas, freight distance, cess calculation, break-even
`test_trends_detection.py`	4	**PASSED**	SMA-7 and SMA-21 momentum indicator classification (RISING/FALLING)
`test_forecasting.py`	2	**PASSED**	Time-series ridge regression, confidence intervals, MAPE metric
`test_weather_intelligence.py`	8	**PASSED**	Open-Meteo REST calls, reverse geocoding, agricultural advisories
TOTAL TEST SUITE	**124**	**100% PASS**	**All 16 test files passed in 191.34s; 0 build errors**

CHAPTER 19: HACKATHON PRESENTATION & VIVA PREPARATION

19.1 Pitch Scripts for the Jury

30-Second Elevator Pitch:
"Judges, smallholder farmers lose thousands of rupees every harvest simply because they sell at the nearest mandi without knowing that a market 40 km away offers 25% higher profit after deducting transportation costs. FARMPRICE AI is an intelligent agricultural platform powered by Snowflake, Open-Meteo, and AGMARKNET. It tells farmers what to plant before sowing and exactly where to sell at harvest to maximize take-home profit."

1-Minute Hackathon Pitch:
"India has over 2,400 APMC mandis, yet smallholder farmers remain trapped in price distress. The core problem is information asymmetry: high quoted prices in major cities are often an illusion once highway freight, loading, and market cess are subtracted. FARMPRICE AI solves this with three pillars: First, an Agronomic Suitability Engine evaluating 43 crops against live weather and soil profiles. Second, a Deterministic Net-Return Engine computing road-freight, APMC cess, and handling fees across candidate mandis to identify the true highest take-home return. Third, an enterprise Snowflake cloud data warehouse managing historical price trends and predictive signals. With full English and Kannada support and 124 passing automated tests, we empower farmers to sell with confidence."

19.2 Forty Comprehensive Viva Questions & Expert Answers

1. Why did you choose Snowflake over PostgreSQL or MongoDB?

Answer: Snowflake decouples storage from compute. When thousands of farmers query multi-year price aggregates across 43 commodities, the compute warehouse scales elastically without impacting ingestion pipelines.

2. What is your Snowflake database architecture?

Answer: FARMPRICE_DB contains four distinct schemas: RAW for raw JSON feeds, STAGING for deduplicated records, AGRICULTURE for transactional tables (MANDI_PRICES, CROP_REQUIREMENTS), and ANALYTICS for curated reporting views.

3. What warehouse size and auto-suspend policy do you use?

Answer: XSMALL warehouse with AUTO_SUSPEND = 60 seconds and AUTO_RESUME = TRUE to eliminate idle cloud compute costs.

4. How do you handle database failures if Snowflake is unreachable?

Answer: The backend implements an automatic fail-safe pattern: if Snowflake network credentials encounter latency or partition, the query manager seamlessly switches to a synchronized local analytical mirror (farmprice_local.db) without crashing.

5. How did you resolve the parameter placeholder divergence in Snowflake?

Answer: SQLite uses '?' placeholders while Snowflake requires '%s'. We implemented dynamic placeholder normalization inside execute_query() in snowflake.py.

6. Where does your market price data come from?

Answer: From the official Indian government portal data.gov.in using the AGMARKNET Daily Prices API resource 9ef84268-d588-465a-a308-a864a43d0070.

7. Are you limited to tomato or single crops?

Answer: No. We dynamically retrieve and map all 43 available Karnataka agricultural commodities, including cereals, pulses, commercial crops, oilseeds, fruits, and vegetables.

8. How do you handle commodity name variations (e.g. 'Paddy(Dhan)')?

Answer: We built a multi-token alias resolution dictionary in crop_recommendation_engine.py that canonicalizes source labels into standard crop entities.

9. How do you prevent duplicate records in your database?

Answer: We construct a unique composite natural key from MD5(market + commodity + variety + arrival_date) and perform an upsert/merge operation.

10. How often is market data refreshed?

Answer: AGMARKNET updates daily after evening auctions; our pipeline runs batch ingestion nightly and caches queries with a 15-minute TTL.

11. What weather provider is integrated?

Answer: Open-Meteo REST API, providing real-time ambient temperature, humidity, precipitation, wind speed, and 7-day agricultural forecasts.

12. What agronomic authority backs your crop requirements?

Answer: Agronomic parameters are calibrated against research publications from the Indian Council of Agricultural Research (ICAR) and UAS Bengaluru.

13. How does weather affect crop recommendations?

Answer: The recommendation engine evaluates upcoming rainfall and temperature against the crop's threshold bounds, penalizing crops susceptible to rot or drought during adverse conditions.

14. How many crops are supported in the suitability engine?

Answer: 43 major Karnataka agricultural crops.

15. What are the three cropping seasons supported?

Answer: Kharif (Monsoon: June–October), Rabi (Winter: October–March), and Summer (January–May).

16. How is road distance estimated?

Answer: We compute the Haversine great-circle distance between coordinates and apply a calibrated 1.25x road curvature factor reflecting rural Indian road geometry.

17. What costs are included in the Net Return formula?

Answer: Road freight transportation, loading/unloading fees (■8/q), and statutory APMC market cess (1.5%).

18. Why does the highest mandi price not guarantee the highest net return?

Answer: Long-distance freight overhead often exceeds the price premium. As demonstrated between Hassan and Bengaluru, a ■150/q price premium was erased by ■8,291 in extra freight.

19. How do you prevent double-counting of costs?

Answer: We separate farm-gate cultivation production costs from post-harvest marketing and transit expenses.

20. How is break-even price calculated?

Answer: Total production expenses plus total selling costs divided by total marketable volume in quintals.

21. Is your price prediction genuine machine learning or rule-based?

Answer: It uses an auto-regressive time-series regression pipeline with 7-day and 14-day lag features and day-of-week seasonality, backed by moving average baselines.

22. How do you evaluate forecast accuracy?

Answer: Using Mean Absolute Percentage Error (MAPE), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE) against historical test holdouts.

23. What happens if historical observations are insufficient?

Answer: If a commodity series has fewer than 14 historical observations, the system explicitly returns INSUFFICIENT_DATA rather than hallucinating prices.

24. What does the Trend Signal indicate?

Answer: It compares SMA-7 against SMA-21. If SMA-7 > SMA-21 + 2.5%, it signals RISING; if below -2.5%, it signals FALLING; otherwise STABLE.

25. Can farmers interpret your predictions as guaranteed prices?

Answer: No. Every report and screen contains prominent disclaimers clarifying that predictions are statistical estimates subject to physical grading and auction dynamics.

26. How did you implement PDF export?

Answer: Using ReportLab in Python, which dynamically compiles vector elements, tables, and branding into lightweight 5KB PDFs in under 50ms.

27. What languages are supported?

Answer: English and Kannada (kn-IN), with instant one-click switching across all navigation, tables, and cards.

28. How do you ensure the report prints cleanly without browser headers?

Answer: We implement comprehensive @media print CSS rules that suppress sidebars, navigation bars, and background clutter while preserving colors with -webkit-print-color-adjust: exact.

29. What is the purpose of the view switcher in Smart Mandi?

Answer: It allows farmers to toggle between the Official Report (Template) view matching official paperwork and the Interactive Card stack for quick scanning.

30. What responsive design techniques are used for mobile farmers?

Answer: Fluid grid layouts, horizontal touch-scrolling data tables, and high-contrast color tokens compliant with WCAG AA accessibility standards.

31. How do you protect database passwords and API tokens?

Answer: All secrets are stored in server-side .env files, read via Pydantic Settings, and excluded from source control via .gitignore.

32. How many automated tests exist in your test suite?

Answer: 124 comprehensive pytest tests covering syncing, recommendations, calculators, trend detectors, and endpoints.

33. What is your API test coverage?

Answer: All 14 FastAPI routers and 16 core integration endpoints are covered with automated HTTP tests in tests/test_audit_endpoints.py.

34. Did the frontend build succeed with zero errors?

Answer: Yes, Vite built 1,499 modules cleanly in 8.06 seconds.

35. How are user sessions maintained?

Answer: Via JWT bearer tokens supporting farmer, yard officer, and system administrator roles.

36. How will illiterate or non-tech-savvy farmers use this app?

Answer: The UI uses visual crop icons, color-coded badges, Kannada voice-friendly audio tags, and large high-contrast buttons.

37. What is the APMC Yard Officer portal used for?

Answer: It allows market committee officers to inspect inbound farmer produce listings and issue verified purchase orders.

38. Can this be deployed on government cloud infrastructure?

Answer: Yes, the containerized FastAPI backend and static Vite frontend can run on NIC / MeghRaj cloud or any Docker/Kubernetes cluster.

39. What is the biggest lesson learned during this project?

Answer: That agricultural decision-making requires marrying hard agronomic constraints with cold financial transportation economics; high market prices alone mean nothing without net-return modeling.

40. If given another month of development, what would you add?

Answer: Direct WhatsApp chatbot integration using Twilio, automated voice call daily price briefings in Kannada, and computer-vision crop grading from smartphone photos.

CHAPTER 20: PRESENTATION DIAGRAMS & SUMMARY

20.1 End-to-End Decision Topology

The architecture unifies five critical dimensions into a single decision surface:
• **Input Dimension:** Farmer coordinates, crop entity, harvest volume.
• **Agronomic Dimension:** ICAR soil, water, and seasonal suitability scores.
• **Meteorological Dimension:** Open-Meteo live atmospheric observations and forecasts.
• **Market Dimension:** AGMARKNET auction prices and arrival tonnages across Karnataka APMCs.
• **Economic Dimension:** Road freight, loading fees, statutory APMC cess, and take-home net profit.

20.2 Executive Project Brief for Hackathon Mentors

FARMPRICE AI represents a fully functional, mathematically rigorous, and cloud-backed agricultural decision platform. It successfully eliminates the 'highest listed price' illusion by providing deterministic net-return modeling, protects against climate volatility through live meteorological advisory, and scales effortlessly on Snowflake cloud data warehousing. With 124 passing automated tests, zero build errors, full English/Kannada bilingual support, and verified ReportLab PDF generation, Team ASTERISK is proud to present a project that directly serves the prosperity of Indian farmers.